Krogh GUI
Comprehensive Mathematical, Numerical, and Implementation Documentation

Technical documentation synchronized to the current code base

23 April 2026

Contents

1 Scope and intent **3**

2 Current code architecture **3**

2.1 Top-level structure 3

2.2 Detailed package map 4

3 Mathematical model **4**

3.1 Geometry and state variables 4

3.2 Blood-gas correction of oxygen affinity 5

3.3 Hemoglobin saturation and differential capacity 5

3.4 Radial Krogh-Erlang profile 5

3.5 Oxygen-dependent metabolic consumption 6

3.6 Capillary axial transport equation 6

4 Numerical methods and algorithm workflow **6**

4.1 Discretization 6

4.2 Capillary ODE integration 6

4.3 2D tissue field with axial diffusion 7

4.4 Outer capillary-tissue coupling 7

4.5 Single-case output metrics 7

5 Inverse reconstruction mathematics **8**

5.1 Problem definition 8

5.2 Optimization strategy in current code . . . 8
5.3 Uncertainty and identifiability additions . . . 9
5.4 Radius-conditioned reconstructions and hidden burden . . . 9

6 Diagnostic model mathematics **9**
6.1 Feature risks . . . 9
6.2 Weighted risk score . . . 9
6.3 State probabilities . . . 10

7 Function-level implementation map **10**
7.1 Forward model core in `krogh_GUI.py` . . . 10
7.2 Reconstruction in `krogh_app/reconstruction.py` . . . 10
7.3 Diagnostics in `oxygenation_diagnostic_mvp.py` . . . 11

8 GUI structure and controls **11**
8.1 Tabs and purpose . . . 11
8.2 Diagnostic tab controls (current implementation) . . . 11
8.3 Numerics tab settings . . . 12

9 Output artifacts: what is generated, where, when **12**
9.1 Interactive outputs . . . 12
9.2 Saved case payloads . . . 12
9.3 Series run bundles . . . 12
9.4 Diagnostic report export . . . 13
9.5 Publication report export . . . 13
9.6 Synthetic validation outputs . . . 13
9.7 Reconstruction benchmark outputs . . . 14

10 Recent extensions now documented **14**

11 Reproducibility and interpretation notes **14**

12 Conclusion **15**

1 Scope and intent

This document is the detailed technical reference for the current Krogh GUI project state. It is written so that a natural scientist can reconstruct what the software computes, how it computes it, and where this logic is implemented in Python.

The documentation is intentionally explicit in five dimensions:

1. mathematical formulation and equation meaning,
2. numerical method and solver workflow,
3. Python implementation mapping (file, function, inputs, outputs, call context),
4. GUI structure and user actions,
5. output files and generated artifacts (what, where, when).

2 Current code architecture

2.1 Top-level structure

Current project-level files and packages relevant to scientific use:

- `app.py`: minimal startup entry point,
- `krogh_GUI.py`: main GUI class and forward model core functions,
- `oxygenation_diagnostic_mvp.py`: probabilistic diagnostic model,
- `krogh_app/`: modular service package,
- `run_synthetic_validation.py`: synthetic validation runner,
- `run_reconstruction_benchmark.py`: reconstruction benchmark runner,
- `tests/`: regression and export tests.

2.2 Detailed package map

Module	Responsibility
`app.py`	Launches `KroghGUI()` and enters Tkinter main loop.
`krogh_GUI.py`	Translation tables, GUI controller methods, forward Krogh solver, single-case computation, series hooks, diagnostic orchestration, export triggers.
`krogh_app/constants.py`	Physical constants, model geometry, discretization defaults, numeric setting bounds and defaults.
`krogh_app/series.py`	Sweep value generation, case grid creation, result row assembly, tighter-tolerance numerics check.
`krogh_app/reconstruction.py`	Inverse fitting (perfusion, mitoP50, metabolic factor), candidate screening, local optimization, post-fit Brent refinement, uncertainty and identifiability summaries, reconstruction plot data.
`krogh_app/diagnostics.py`	Wrapper around `oxygenation_diagnostic_mvp`.
`krogh_app/persistence.py`	Case save/load payloads, diagnostic report generation, publication report text/TEX/PDF generation and file writing.
`krogh_app/validation.py`	Synthetic validation suites, trend checks, robustness checks, CSV/JSON report saving.
`krogh_app/benchmarking.py`	Multi-case reconstruction benchmark, optimized vs legacy-grid comparison, CSV/JSON report saving.
`krogh_app/plotting.py`	Shared plotting styles and rendering helpers.
`krogh_app/ui/*.py`	Tkinter layout, controls, dialogs, runtime status, execution coordination, figure and bundle export helpers.

3 Mathematical model

3.1 Geometry and state variables

The model uses a Krogh-cylinder representation:

- capillary radius: $R_{cap} = 5 \times 10^{-4}$ cm,
- tissue radius: $R_{tis} \in \{30, 50, 100\} \times 10^{-4}$ cm (default base geometry: 100×10^{-4} cm),
- capillary length: $L_{cap} = 0.1$ cm.

In the GUI, the base tissue radius is selectable as 30, 50, or 100 μm. The selected value is applied globally to the forward-model geometry (radial grid, radial averaging weights, and all single/series simulations), while the diagnostic radius-sensitivity comparison remains available as an additional independent reconstruction analysis.

Unknown fields:

- capillary oxygen partial pressure along axis: $P_c(z)$,

- tissue oxygen field: $P(z, r)$.

3.2 Blood-gas correction of oxygen affinity

The code computes an effective hemoglobin half-saturation pressure:

$$P_{50,eff} = P_{50} \cdot 10^{\Delta}, \quad \Delta = \beta_{pH}(pH - PH_{ref}) + \beta_{CO_2} \log_{10}(pCO_2/PCO2_{ref}) + \beta_T(T - T_{ref}).$$

Constants in `krogh_app/constants.py`:

- $P_{50} = 26.8$ mmHg,
- $\beta_{pH} = -0.48$,
- $\beta_{CO_2} = 0.06$,
- $\beta_T = 0.024$,
- $PH_{ref} = 7.40$, $PCO2_{ref} = 40$ mmHg, $T_{ref} = 37°$C.

Interpretation in plain language: this step shifts the oxygen dissociation curve depending on acidity, CO_2, and temperature. A shifted curve changes how strongly pressure drops for a given extraction level.

3.3 Hemoglobin saturation and differential capacity

Hill saturation:

$$S(P) = \frac{P^n}{P^n + P_{50,eff}^n}, \quad n = 2.7.$$

Differential oxygen capacity used in the axial capillary ODE:

$$\frac{dC}{dP} = C_{Hb}\frac{dS}{dP} + \alpha,$$

with $C_{Hb} = 0.20$ and $\alpha = 3 \times 10^{-5}$ in current implementation units.

Meaning: the same extraction flux causes a larger pressure drop where dC/dP is small (flatter dissociation segment), and a smaller pressure drop where dC/dP is large.

3.4 Radial Krogh-Erlang profile

At fixed z and effective metabolic demand M, the radial approximation is

$$P(r) = P_c + \frac{M}{4K}(r^2 - R_{cap}^2) - \frac{MR_{tis}^2}{2K} \ln\left(\frac{r}{R_{cap}}\right),$$

with diffusion parameter $K = 2 \times 10^{-9}$.

This profile is used as a direct local estimate and as initialization for the 2D iterative field solve.

3.5 Oxygen-dependent metabolic consumption

The code uses a Michaelis-Menten-like saturation law with a nonzero floor:

$$M(P) = M_{max}\left[f_{min} + (1 - f_{min})\frac{P}{P + P_{half}}\right],$$

where $f_{min} = 0.05$ and P_{half} is the mitochondrial half-saturation parameter (fit parameter in inverse mode).

Meaning in plain terms: metabolism decreases at low oxygen but never to exact zero. This avoids non-physiological collapse and improves numerical stability.

3.6 Capillary axial transport equation

The axial capillary equation solved numerically is

$$\frac{dP_c}{dz} = -\frac{M_{eff}(z)\,\pi R_{tis}^2}{Q_{local}\,dC/dP},$$

with

$$Q_{local} = \max(\text{perfusion_factor} \cdot Q_{flow}, 10^{-12}).$$

The M_{eff} term is obtained from iterative coupling to tissue oxygenation (without axial tissue diffusion) or from the 2D coupled field (with axial tissue diffusion).

4 Numerical methods and algorithm workflow

4.1 Discretization

Default mesh (from `krogh_app/constants.py`):

- axial points: $NZ = 80$,
- radial points: $NR = 80$,
- `z_eval = linspace(0, L_cap, NZ)`,
- `r_vec = linspace(R_cap, R_tis, NR)`.

Radial averaging uses area-aware weighting `radial_weights`.

4.2 Capillary ODE integration

Library method: `scipy.integrate.solve_ivp(..., method="RK45")` with defaults:

- `rtol = 1e-9`,
- `atol = 1e-11`,
- `max_step = dz`.

4.3 2D tissue field with axial diffusion

If axial diffusion is enabled, `solve_tissue_field_with_axial_diffusion(...)` iterates a finite-difference update over the (z, r) grid with:

- inner iteration cap: `AXIAL_DIFFUSION_MAX_ITER = 140`,
- convergence criterion: `max(abs(P - P_old)) < 1e-5`,
- relaxation factor: `AXIAL_DIFFUSION_RELAX = 0.45`.

Boundary treatment in code:

- capillary boundary fixed to current `P_c_axial`,
- no-flux outer radial boundary via copy of neighbor node,
- axial end handling by nearest-neighbor copy.

4.4 Outer capillary-tissue coupling

`solve_axial_capillary_po2(...)` alternates between tissue-field update and capillary ODE re-integration:

1. solve tissue field from current `P_c_axial`,
2. compute slice-wise metabolic demand from tissue field,
3. re-solve capillary ODE with interpolated demand,
4. stop if capillary-profile change is below `AXIAL_COUPLING_TOL = 1e-4` or max iterations reached.

This operator-splitting approach is transparent and robust for the intended workflow.

4.5 Single-case output metrics

`run_single_case(...)` computes and returns:

- P_{venous} (capillary terminal PO_2),
- $P_{tissue,min}$,
- $P_{tissue,p05}$ (5th percentile of all tissue nodes),
- hypoxic fractions below 1, 5, 10 mmHg,
- fractions above absolute thresholds (default 100 and 200 mmHg, plus optional extras),
- fractions above relative thresholds (defaults 90, 50, 30% of chosen reference),

- relative reference key and value (`inlet` or `tissue_max`),
- sensor-like average PO_2 (weighted radial, then axial mean),
- arterial and venous saturations,
- flow in nL/s.

5 Inverse reconstruction mathematics

5.1 Problem definition

Given measured/entered values from diagnostics:

- blood-gas PO_2, pCO_2, pH, temperature,
- sensor target PO_2,
- optional venous saturation (converted to venous PO_2 target),

fit a plausible tuple

$$(\text{perfusion_factor}, P_{half}, \text{metabolic_rate_rel})$$

that reproduces sensor and venous targets.

5.2 Optimization strategy in current code

Main API: `KroghReconstructor.fit_joint_parameters(...)` in `krogh_app/reconstruction.py`.

Current default strategy: `search_strategy="optimized"`.

High-level workflow:

1. Build geometric candidate grids (perfusion, P_{half}, metabolic factor).
2. Screening pass to keep promising candidates.
3. Local optimization around best screening seed.
4. Probe neighborhood near best candidate.
5. Post-fit two-stage 1D `brentq` refinement to enforce sensor target:
   - stage 1: metabolic-rate refinement first,
   - stage 2: perfusion refinement only if needed.
6. Attach uncertainty and identifiability summaries.

Alternative mode: `legacy_grid` (larger exhaustive candidate evaluation).

5.3 Uncertainty and identifiability additions

Recent extensions implemented in `krogh_app/reconstruction.py`:

- `_bootstrap_uncertainty(...)`: repeated perturbed-target refits,
- `_estimate_identifiability(...)`: near-optimal set conditioning summary,
- fit warning logic includes large sensor/venous mismatch and wide uncertainty indicators.

5.4 Radius-conditioned reconstructions and hidden burden

During reconstruction plot-data construction:

- hidden hypoxic burden map includes thresholds below 1, 5, 10, 15, 20 mmHg,
- radius scenarios are generated for 30, 50, and 100 µm assumptions,
- summary text reports radius sensitivity and assumption-dependence.

6 Diagnostic model mathematics

The diagnostic layer in `oxygenation_diagnostic_mvp.py` uses explicit feature transforms.

6.1 Feature risks

Each major feature is mapped by a sigmoid or derived transform into $[0, 1]$:

- low blood-gas PO_2 risk,
- low tissue/sensor PO_2 risk,
- acidosis risk,
- hypercapnia risk,
- anemia risk,
- low venous saturation risk,
- temperature stress risk,
- arterial-to-sensor gap risk.

6.2 Weighted risk score

Current weight set in code:

$$R = 0.22R_{po2} + 0.24R_{sensor} + 0.08R_{gap} + 0.12R_{acid} + 0.10R_{CO2} + 0.08R_{Hb} + 0.06R_{venous} + 0.02R_T.$$

Score is clamped to $[0, 1]$ and then hard-floor overrides apply for very low oxygen values.

6.3 State probabilities

Five states are scored using Gaussian-like PO_2-band centers plus risk/gap modifiers, then normalized with softmax:

- normoxia,
- intermediate oxygenation,
- low oxygenation approaching critical,
- hypoxia,
- profound hypoxia.

This is probabilistic, not a single hard-threshold classifier.

7 Function-level implementation map

7.1 Forward model core in `krogh_GUI.py`

Function	Parameters (main)	Returns	Called from
`effective_p50`	pH, pco2, temp_c	float p50_eff	single-case run, diagnostics/reconstruction prep
`hill_saturation`	P, optional p50_eff	saturation fraction	output metrics, venous conversion
`dC_dP`	P, optional p50_eff	differential capacity	capillary ODE RHS
`krogh_erlang`	r, P_c, M, K, R_c, R_t	radial PO2 profile	initialization, fixed-point demand update
`michaelis_menten_consumption`	P_local, P_half, M_max	local consumption	demand update, tissue iteration
`effective_consumption_from_capillary_po2`	P_c, P_half, metabolic_rate_rel, max_iter, tol	scalar effective demand	initial capillary solve and no-axial branch
`solve_capillary_profile`	ODE callback, P_inlet	axial PO2 vector	initial and coupled capillary solve
`solve_initial_capillary_po2`	P_inlet, P_half, p50_eff, perfusion, metabolic_rel	initial capillary profile	main coupled solver
`solve_tissue_field_with_axial_diffusion`	P_c_axial, P_half, metabolic_rel, initial_guess	2D tissue field	coupled solver
`solve_axial_capillary_po2`	P_inlet, P_half, p50_eff, include_axial, perfusion, metabolic_rel	capillary profile, tissue field, axial demand	single-case and inverse reconstruction simulations
`run_single_case`	full physiological and threshold inputs	results dict	single-run button, sweep rows, validation helpers

7.2 Reconstruction in `krogh_app/reconstruction.py`

Key public methods:

- `fit_joint_parameters(...)`: main inverse fit,
- `build_plot_data(...)`: constructs reconstruction output payload for visualization/reporting.

Key private methods invoked internally:

- candidate screening/evaluation helpers,
- local optimization helper,
- `_refine_candidate_to_sensor_target` (two-stage `brentq`),
- `_bootstrap_uncertainty`,
- `_estimate_identifiability`,
- radius scenario construction and summaries.

7.3 Diagnostics in `oxygenation_diagnostic_mvp.py`

Main public functions:

- `alert_decision(oxygen_input, yellow_threshold, orange_threshold, red_threshold)`,
- `effective_p50`, `hill_saturation`.

Return payload includes risk score, alert level, confidence/certainty, five state probabilities, feature-risk map, dominant drivers, and normalized input echo.

8 GUI structure and controls

Main class: `KroghGUI(tk.Tk)` with window defaults `1080x860` and minimum `980x720`.

8.1 Tabs and purpose

- Inputs: single-case physiological parameters (including perfusion factor, relative metabolic rate, and selectable tissue radius) and threshold settings.
- Series: 1D/2D sweeps, plot mode selection, optional save bundle.
- Numerics: solver and bootstrap settings.
- Diagnostic: blood-gas plus sensor interpretation, reconstruction, report export, benchmark launch.

8.2 Diagnostic tab controls (current implementation)

Controls include:

- entries for PO_2, pCO_2, pH, temperature, sensor PO_2, hemoglobin, venous saturation,
- threshold entries for yellow/orange/red risk cutoffs,
- radius mode combo: `all variants` or `selected radius only`,
- selected radius combo: 30, 50, 100 µm,
- buttons: use single-case values, run diagnostic, save template, save diagnostic report, save publication report, reconstruct Krogh cylinder, run reconstruction benchmark,
- checkbutton: auto-save independent 30/50/100 µm reconstructions to `Diagnostic reports`.

8.3 Numerics tab settings

User-adjustable numeric fields exposed through constants-backed specs:

- ODE: `ode_rtol`, `ode_atol`, `ode_max_step`,
- tissue iteration: `axial_diffusion_max_iter`, `axial_diffusion_tol`,
- coupling iteration: `axial_coupling_max_iter`, `axial_coupling_tol`,
- reconstruction uncertainty: `bootstrap_samples`.

9 Output artifacts: what is generated, where, when

9.1 Interactive outputs

- main output text panel after single-case and series runs,
- diagnostic output text panel after diagnostic/reconstruction actions,
- matplotlib figures for 2D/3D/heatmap displays.

9.2 Saved case payloads

Action: `Save case` button.

Output: JSON payload storing model entries, diagnostic entries, numerics, UI mode state, and cached last results.

9.3 Series run bundles

Action: Series run with save-bundle option.

Created by `krogh_app/ui/figures.py`:

- folder name: `krogh_series_run_YYYYMMDD_HHMMSS` (with suffix if needed),
- `series_results.xlsx` and `Series_Setup` sheet (or CSV fallback if `openpyxl` unavailable),
- saved plot PNG files for selected fields,
- `run_parameters.txt`.

9.4 Diagnostic report export

Action: `Save diagnostic report`.

Outputs:

- JSON (default) or CSV,
- contains input data, diagnostic result, reconstruction payload flattening, uncertainty and hypoxic map fields.

9.5 Publication report export

Action: `Save English publication report`.

Formats:

- TXT,
- TEX,
- PDF (via `pdflatex`, with automatic intermediate TEX generation).

Content source: `CaseRepository.build_publication_report_text/tex`.

If reconstruction figures are available, they are embedded in TEX via `\includegraphics{\detokenize{...}}` to robustly handle file paths.

9.6 Synthetic validation outputs

Script: `run_synthetic_validation.py`.

Default output folder: `Diagnostic reports/`.

Generated files:

- `synthetic_validation_report.csv/json`,
- `synthetic_validation_trend_checks.csv/json`,
- `synthetic_validation_robustness.csv/json`.

9.7 Reconstruction benchmark outputs

Script: `run_reconstruction_benchmark.py`.

Default output folder: `Diagnostic reports/`.

Generated files:

- `reconstruction_benchmark.csv/json`,
- `reconstruction_benchmark_comparison.csv/json`.

10 Recent extensions now documented

The following additions are explicitly integrated in this revision:

1. inverse reconstruction uncertainty via bootstrap refits,
2. identifiability summary from near-optimal neighborhoods,
3. hidden hypoxic burden below 1/5/10/15/20 mmHg,
4. radius-conditioned interpretations for 30/50/100 µm,
5. independent radius reconstruction figure export option,
6. publication report pipeline (TXT/TEX/PDF) with figure embedding,
7. synthetic validation suite outputs,
8. reconstruction benchmark with optimized vs legacy-grid comparison,
9. full numerics-tab controls and persistence integration.

11 Reproducibility and interpretation notes

- A fit is model-conditional: it is the best explanation inside current geometry/physiology assumptions.
- Boundary hits and wide uncertainty indicate weaker parameter separability.
- Radius sensitivity should be interpreted as assumption dependence, not as direct morphological measurement.
- Solver settings can materially affect difficult edge cases; numerics checks should be used in sweep studies.

12 Conclusion

The current Krogh GUI implementation is a transparent, auditable chain from physiology-inspired equations to numerical solution, probabilistic diagnostics, inverse reconstruction, and exportable reporting. This document now maps those steps explicitly to the code so that results can be reproduced and independently reviewed.